Niklas Glaser

Heinrich-Heine-Universität Düsseldorf

April 2026

Overview

Goals and Research Question

Goal

- improve the Base Agent with League Training (LT)
- training against a more diverse set of opponents
- focus on robustness and generalization

Base Agent

- combination of Behavioral Cloning (BC) and Proximal Policy Optimization (PPO)
- strong baseline, but weaknesses against several opponents

Research Question

- How much does LT with PPO improve robustness and generalization of the BC/PPO-pretrained agent?
- How strongly does this depend on the diversity of initial agents and BC experts?

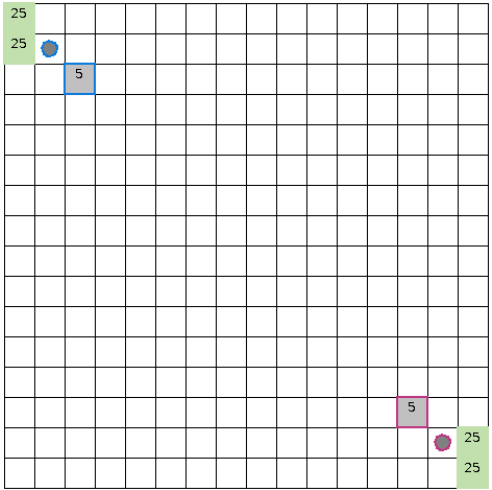
Setting

MicroRTS

- simplified RTS environment for RL research
- still contains resource management, unit production and combat
- supports controlled and repeatable experiments

Main RL challenges

- large combinatorial action space
- simultaneous decisions
- delayed and sparse rewards
- robustness against diverse opponents



▸ Open MicroRTS Example Video

Training Pipeline

BC and BC-FT

- imitate experts
- BC-FT: only winning replays

PPO

- iteratively update policy
- refine the policy through exploration

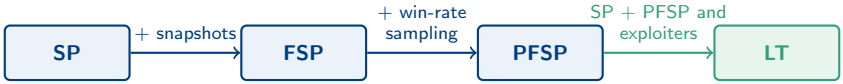
LT

- train against an evolving opponent population
- add exploiters

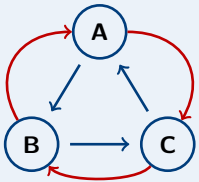
PPO fine-tuning

- repair the main remaining weakness after LT
- add WorkerRushAI as opponent

From SP to LT



cyclical learning of SP



Blue arrows: learning direction
Red arrows: policies the current policy performs well against

Why LT helps

- improves stability by identifying and exploiting weaknesses
- creates diverse set of opponents without the main agent having to find them through exploration

LT Design

Main Agent

- primary policy
- resulting agent

Historical Agents

- frozen checkpoints kept in the opponent pool
- created by all other league agent types

Main Exploiters

- exploit weaknesses of the Main Agents
- resets, when checkpointing

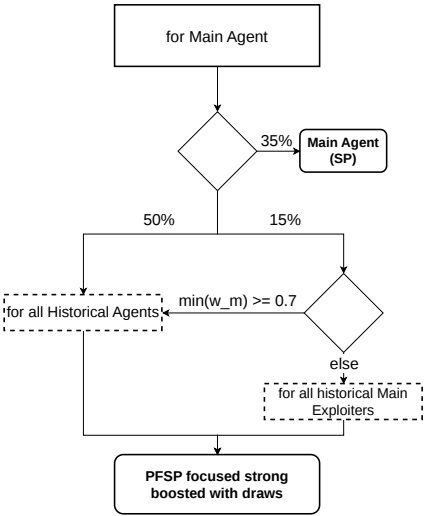
League Exploiters

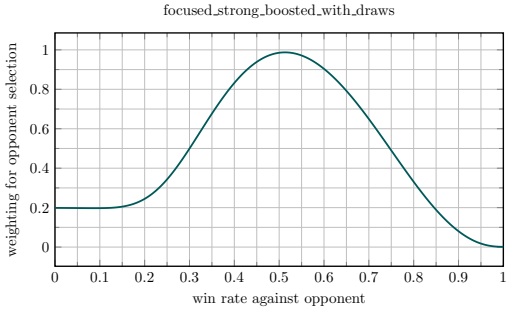
- exploit broader weaknesses across the league
- has a chance of resetting when checkpointing

Key idea

LT turns the robustness against diverse opponents into part of the optimization target.

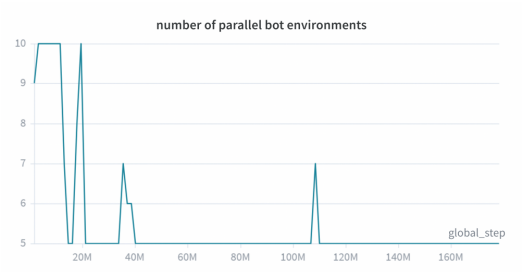
LT Main Agent Matchmaking



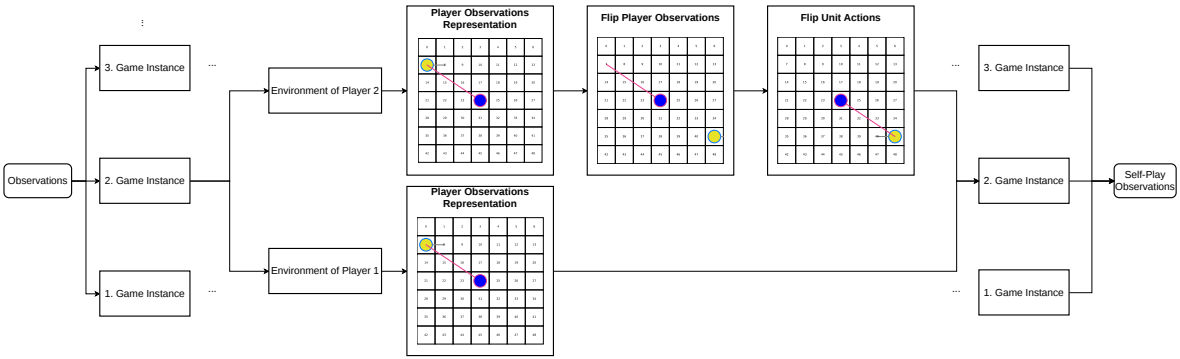


Bot Environments

- additional dynamic bot environments (CoacAI and Mayari)
- scripted bots: CoacAI, Mayari, WorkerRushAI, PassiveAI, LightRushAI, RandomAI and RandomBiasedAI
- MCTS based bots: Rojo, MixedBot, Izanagi, Tiamat, Droplet, GuidedRojoA3N and NaiveMCTS



Observation Adjustment for SP



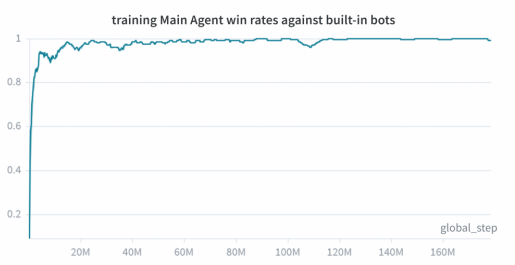
Player 2 observations are transformed so SP remains compatible with a Base Agent only trained as Player 1.

Other Implementation Changes

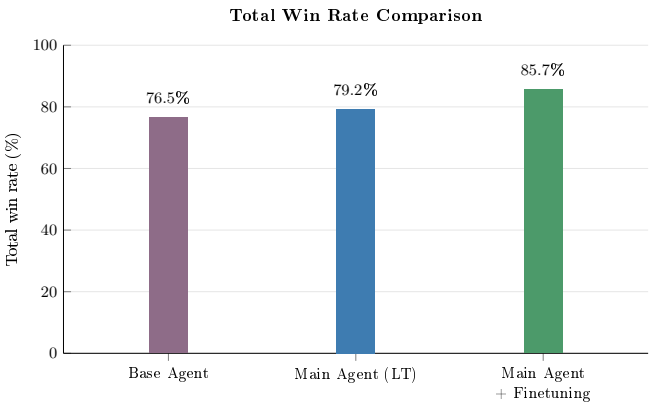
- delta score
- annealed entropy coefficient
- new initial agents

Experiment Setup

- map: `basesWorkers16x16A`
- LT setup: 1 Main Agent, 4 Main Exploiters, 1 League Exploiter
- 25 environments per learning agent
- 5 to 10 additional dynamic bot environments
- approximately 130 million LT steps
- training time: about 100 hours
- maximum setup size: 160 parallel environments
- evaluations over at least 100 games against 12 opponents after LT and after fine-tuning



Main Quantitative Result



- overall loss rate drops from **7.7%** to **2.2%**

Evaluation Results

Opponent	Main Agent without new initial agents	Base Agent	Main Agent (LT)	Main Agent + Fine-tuning
bots used in training				
CoacAI	**100.0**	96.0	**100.0**	**100.0**
Mayari	**100.0**	67.3	**100.0**	**100.0**
scripted bots				
WorkerRushAI	31.0	**100.0**	35.0	99.5
PassiveAI	**100.0**	99.7	**100.0**	**100.0**
LightRushAI	**100.0**	98.0	**100.0**	**100.0**
RandomAI	**100.0**	99.0	**100.0**	**100.0**
RandomBiasedAI	91.0	79.0	97.0	**97.5**
MCTS based bots				
Rojo	**100.0**	81.0	**100.0**	**100.0**
MixedBot	45.0	43.3	57.0	**60.5**
Izanagi	68.0	71.7	70.0	**72.5**
Tiamat	94.0	93.0	**99.0**	**99.0**
Droplet	45.0	67.7	62.0	**75.5**
GuidedRojoA3N	64.0	74.7	87.0	**88.5**
NaiveMCTS	0.0	0.7	2.0	**6.5**
overall	74.1	76.5	79.2	**85.7**

Evaluation Results

Opponent	Main Agent without new initial agents	Base Agent	Main Agent (LT)	Main Agent + Fine-tuning
bots used in training				
CoacAI	**0.0**	2.0	**0.0**	**0.0**
Mayari	**0.0**	27.3	**0.0**	**0.0**
scripted bots				
WorkerRushAI	63.0	**0.0**	55.0	0.5
PassiveAI	**0.0**	**0.0**	**0.0**	**0.0**
LightRushAI	**0.0**	2.0	**0.0**	**0.0**
RandomAI	**0.0**	**0.0**	**0.0**	**0.0**
RandomBiasedAI	**0.0**	**0.0**	**0.0**	**0.0**
MCTS based bots				
Rojo	**0.0**	5.3	**0.0**	**0.0**
MixedBot	33.0	34.3	20.0	**13.5**
Izanagi	15.0	16.3	7.0	**6.0**
Tiamat	6.0	7.0	1.0	**0.5**
Droplet	22.0	**9.0**	22.0	10.5
GuidedRojoA3N	**0.0**	1.7	**0.0**	**0.0**
NaiveMCTS	**0.0**	2.7	**0.0**	**0.0**
overall	9.9	7.7	7.5	**2.2**

Interpretation of the Results

hypothesis:

- LT turns robustness against diverse opponents into part of the optimization target
- LT helps to find a better local optimum that is then further improved by fine-tuning, while pure PPO often gets stuck in not as generalized local optima.

Player 1 Priority

Observation

MicroRTS gives player 1 action priority, so a potential concern is that self-play style training could accidentally favor player-1-specific behavior.

Result from the thesis

In one self-match evaluation of the same agent over 200 games, player 1 achieved 38% wins, 28% draws and player 2 achieved 34% wins.

- only small effect on evaluations
- alternating the learning side to prevent the agent from exploiting player-1-specific priority

Open Player 1 Priority Example Video

Main Limitation: Diversity

▸ Open Base Agent Example Video

Main Limitation: Diversity

- Exploiters initialized from similar policies tend to discover similar weaknesses.
- Some strategically different exploits are hard to reach because they require first unlearning the Base Agent's preferred behavior.
- New initial agents with focuses on different unit types to improve exploiter diversity and Main Agent generalization.
- Hypothesis: LT therefore depends strongly on the diversity already present at initialization

Remaining Limitations and Future Work

- thesis is limited to one map: `basesWorkers16x16A`
- the setting uses full observability and no fog of war
- high computational cost
- investigate stronger diversity mechanisms

Conclusion

Conclusion

LT with PPO improves the robustness and generalization of the MicroRTS Base Agent. Additional PPO fine-tuning improves the final agent even further.

Key takeaway

The effectiveness of LT depends strongly on exploiter diversity, which in turn depends on the diversity of the initial agents and BC experts.

- LT improves most matchups and the overall performance
- fine-tuning further improves the final agent
- diversity of initial agents is the critical practical factor

Thank you

Questions?

Main Agent vs League Exploiter

- League Exploiter resets
- different Hyperparameters
- Historical League Exploiter are not included as Main Exploiter opponents.

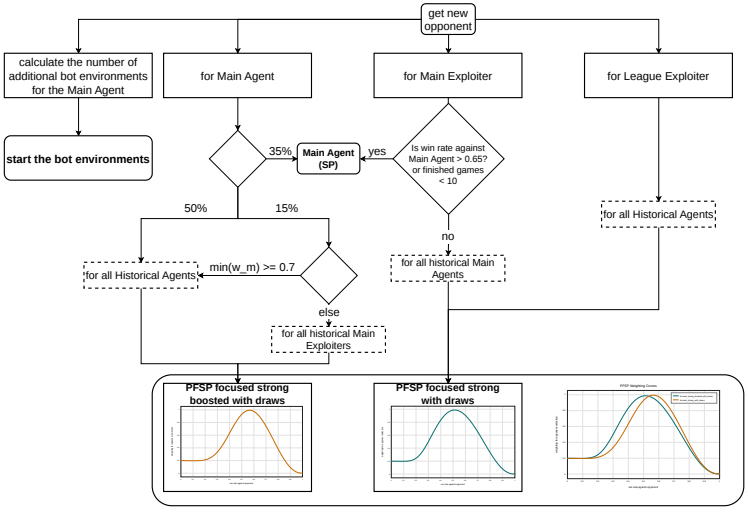
Why adjust the observations for SP?

- Gym-MicroRTS: Toward affordable full game real-time strategy games research with deep reinforcement learning implemented SP and concluded that a likely reason for worse performance of SP was that SP also has to train, to start as Player 2, while pure PPO only trained as Player 1.
- difficulties retrainig the Base Agent.

Why are the win rates against NativeMCTS low?

- it is one of the only opponents that give up winning the game and focus on staying far away from all opponents.
- This results in many draws and few losses.

Full LT Matchmaking



Matchmaking and opponent sampling in the LT setup.

New Initial Agents

1 agent per unit type

Heavy focused

- Base Agent

Light focused

- 9 epochs of BC with LightRushAI as the expert
- opponents for BC: mayari, workerRushAI, lightRushAI

Worker focused

- 8 epochs of BC with WorkerRushAI as the expert
- opponents for BC: mayari, workerRushAI, lightRushAI

Ranged focused

- 6 epochs of BC with CoacAI as the expert
- 180 updates of PPO against the Base Agent, CoacAI and Mayari
- opponents for BC: workerRush, lightRushAI, mayari, coacAI

Why Not Less Environments?

- Environments effect rollout size
- smaller rollouts make PPO updates unstable, because of less knowladge of where to update to
- More steps per rollout have the same problems as more environments.